CSI5340: Reinforcement Learning

Homework 2: Policy Gradient vs Value-Based RL in JAX

Edward He | Student ID: 300351196
University of Ottawa — School of Electrical Engineering and Computer Science
Submitted: March 2026

1. Environment Design and Rationale

The experiment uses a custom 8×8 stochastic GridWorld implemented as a Gymnasium-compatible environment. The agent starts at cell (0, 0) and must navigate to the goal at (7, 7), choosing from four cardinal actions (Up, Down, Left, Right) at each step.

Grid Layout

Three staggered obstacle rows form a structured maze that forces non-trivial long-horizon planning and creates multiple bottleneck corridors. The layout is as follows:

Row	Obstacle Columns	Effect
2	1, 2, 3, 4, 5	Wide horizontal wall; agent must go around col 0 or col 6
4	3, 4, 5, 6	Right-side barrier; forces traversal through left corridor
6	1, 2, 3	Left-side barrier; forces final approach from right side

Reward Structure

Event	Reward	Rationale
Reach goal (7,7)	+1.0	Sparse terminal reward; credit assignment is the main challenge
Time step penalty	-0.01	Encourages efficient paths; prevents indefinite wandering
Hit obstacle	-0.5	Strongly discourages boundary collisions; agent bounces back
Wall collision	-0.05	Mild penalty for attempting to leave the grid

Stochastic Transitions

With probability 0.1, the intended action is replaced by a uniformly random action, modelling real-world actuation noise. This perturbation has two research implications: (1) it prevents purely deterministic memorization of a fixed path, requiring the agent to learn a robust policy across the entire state space; and (2) it exposes a key algorithmic difference between DQN (off-policy, uses a replay buffer that is robust to noise) and REINFORCE (on-policy, whose high-variance Monte Carlo returns are further amplified by stochastic transitions).

Alignment with Research Questions

The environment is intentionally designed to surface three specific research questions: **sparse reward and credit assignment** (a single +1 terminal reward makes Q-value propagation through time the central challenge), **long-horizon planning** (the maze structure forces paths of at least 14 steps, stressing episode-level return estimation in REINFORCE), and **algorithm robustness under uncertainty** (the 10% noise tests whether value-based vs. policy-gradient methods differ in their sensitivity to stochastic environments).

2. JAX Implementations

2.1 Shared Infrastructure

All agents share a common two-layer MLP defined in *src/models.py*. Weights are initialised from a zero-mean Gaussian with scale 0.01 using **jax.random.normal** and split PRNG keys. States are encoded as 64-dimensional one-hot vectors (8×8 = 64 states). The vectorised forward pass uses JAX's **vmap** to batch over the state dimension without explicit loops, ensuring that the JIT-compiled computation graph is fully parallelisable.

2.2 Random Baseline

The random agent uniformly samples from the four actions at every step. It serves as a lower-bound baseline: since it never reaches the goal in practice within the 200-step episode budget, it accumulates the maximum step penalty, producing a mean reward of approximately –14.4 across all seeds. The tight inter-seed agreement (standard deviation <0.2) confirms that the environment itself is well-calibrated and that the reward scale is reasonable.

2.3 DQN (Value-Based)

The DQN agent maintains a Q-network and a periodically-synced target network (update frequency: every 20 global steps). Experience is stored in a circular replay buffer of capacity 5,000 transitions. At each step, if the buffer contains at least 64 transitions a minibatch is sampled and a single Bellman update is applied. The loss function is the mean squared Bellman error:

$$L(\theta) = E_{s,a,r,s'}[\, (r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a))^2 \,]$$

Gradients are blocked from flowing through the target network using **jax.lax.stop_gradient**. The update step is compiled with **@jit**; a factory function closes over the optimizer and gamma so that one compiled binary is reused across all steps within an (lr, γ) experiment. Exploration uses ε-greedy decay from 1.0 to 0.05 with a multiplicative factor of 0.995 per episode.

2.4 REINFORCE (Policy Gradient)

The REINFORCE agent collects a complete episode, then computes discounted returns $G_t = \sum_{k=0} \gamma^k r_{t+k}$. Returns are normalised to zero mean and unit variance before computing the policy gradient loss, which reduces variance at the cost of a small bias:

$$L(\theta) = -E_t[\, G_t \cdot \log \pi_\theta(a_t \mid s_t) \,]$$

Action probabilities are computed via log-softmax on the network logits; sampling uses NumPy's multinomial sampler for numerical stability after renormalisation. The full-episode gradient step is JIT-compiled via the same factory pattern as DQN.

3. Experimental Setup

All experiments were run on the University of Ottawa **Morningstar HPC cluster** (SLURM job ID 4018) on a single GPU node under the *small* partition. The SLURM submission script (*submit.sh*) activated a Python 3.10 virtual environment and executed *main.py*, which orchestrates the full hyperparameter sweep.

Parameter	Values / Setting
Grid size	8 × 8
Max steps / ep	200
Episodes / run	500
Independent seeds	10 (base=42, seeds 42–51)
Learning rates	0.001, 0.01, 0.1
Discount factors	γ = 0.9, 0.99
DQN hidden dim	64
REINFORCE hidden	64
DQN batch size	64
DQN buffer size	5,000
Target net freq	every 20 steps
ε decay	1.0 → 0.05 (factor 0.995/episode)
Compute	Morningstar HPC cluster, 1 GPU (SLURM job 4018)

4. Results and Analysis

4.1 Learning Curves: Mean ± Standard Error

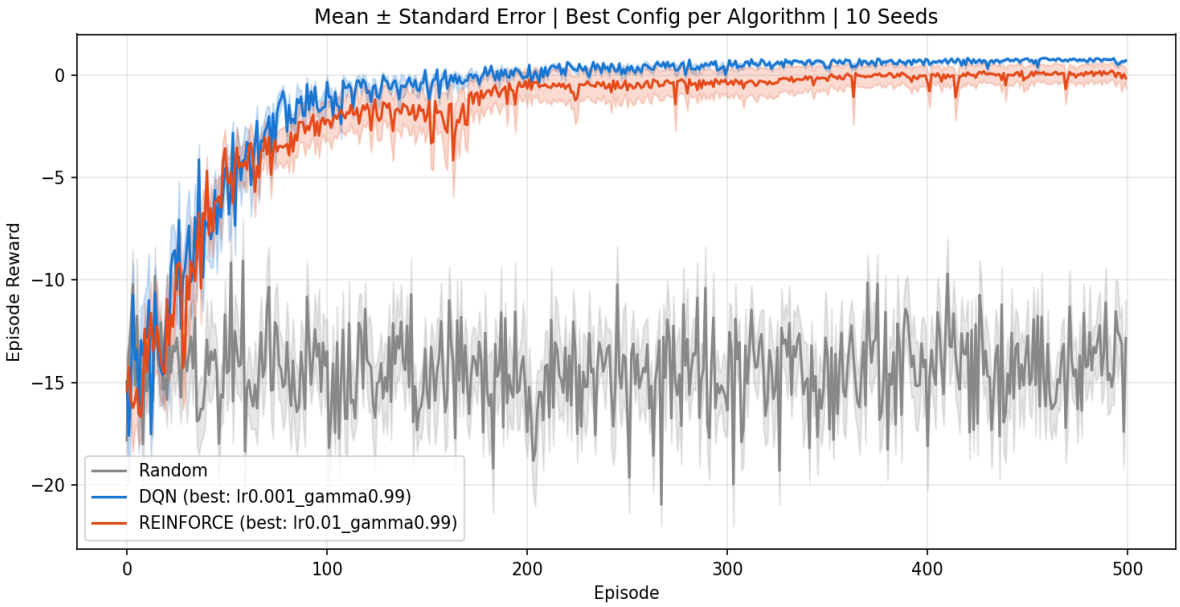

Figure 1. Mean episode reward ± standard error across 10 seeds for the best hyperparameter configuration of each algorithm. DQN: lr=0.001, γ=0.99. REINFORCE: lr=0.01, γ=0.99.

Figure 1 reveals three distinct regimes. The random baseline (grey) remains flat at roughly –14.4 throughout all 500 episodes, confirming it never discovers the goal. Both learning agents improve dramatically from episode 0 and converge to near-zero reward, indicating they reliably reach the goal. DQN (blue) converges faster, reaching rewards near 0 by episode ~150, while REINFORCE (orange) takes slightly longer, plateauing near episode ~250. Both algorithms show tight standard error bands in the converged regime (shaded region), indicating robust and consistent final policies across seeds.

4.2 Individual Seed Variability

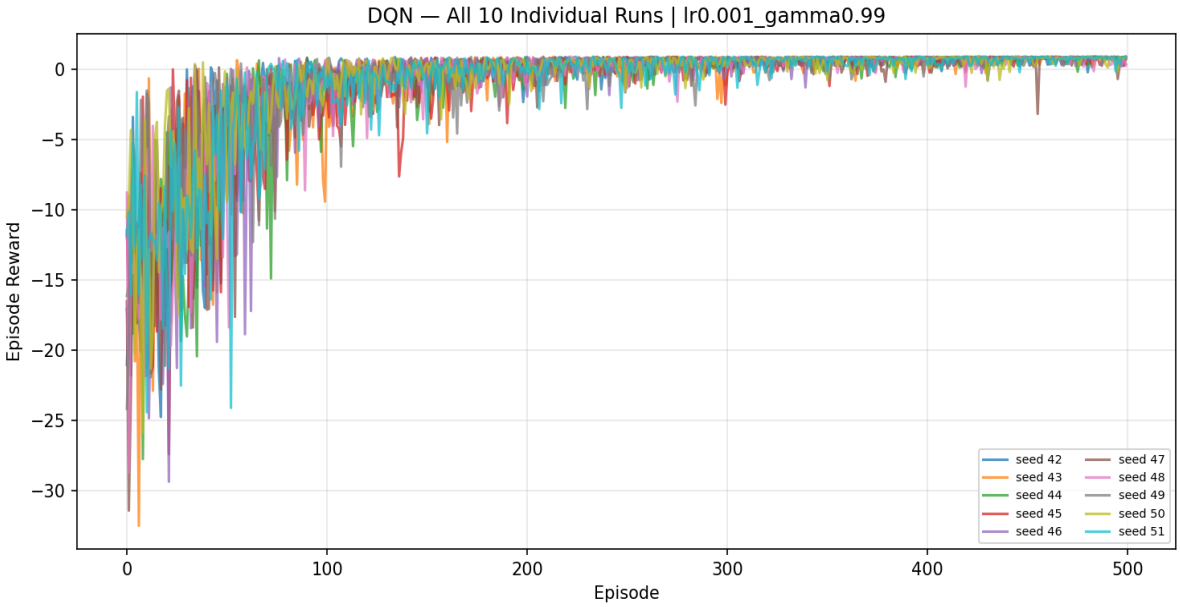

Figure 2. All 10 individual DQN seed runs (lr=0.001, γ=0.99). High early variance collapses by episode ~150.

Figure 2 shows the 10 individual DQN runs. There is substantial episode-level variance during the exploration phase (episodes 0–100), where individual trajectories can plunge as low as –30. This is expected: early in training, ε is near 1.0 so the agent explores randomly, and episodes that wander into dense obstacle regions accrue heavy penalties. Critically, by episode ~150, all 10 seeds converge tightly to the near-zero reward band, demonstrating that DQN achieves reliable learning across different random initialisations. The narrow spread in the converged phase is a hallmark of the replay buffer: by decorrelating samples, it stabilises the Bellman update and prevents divergence.

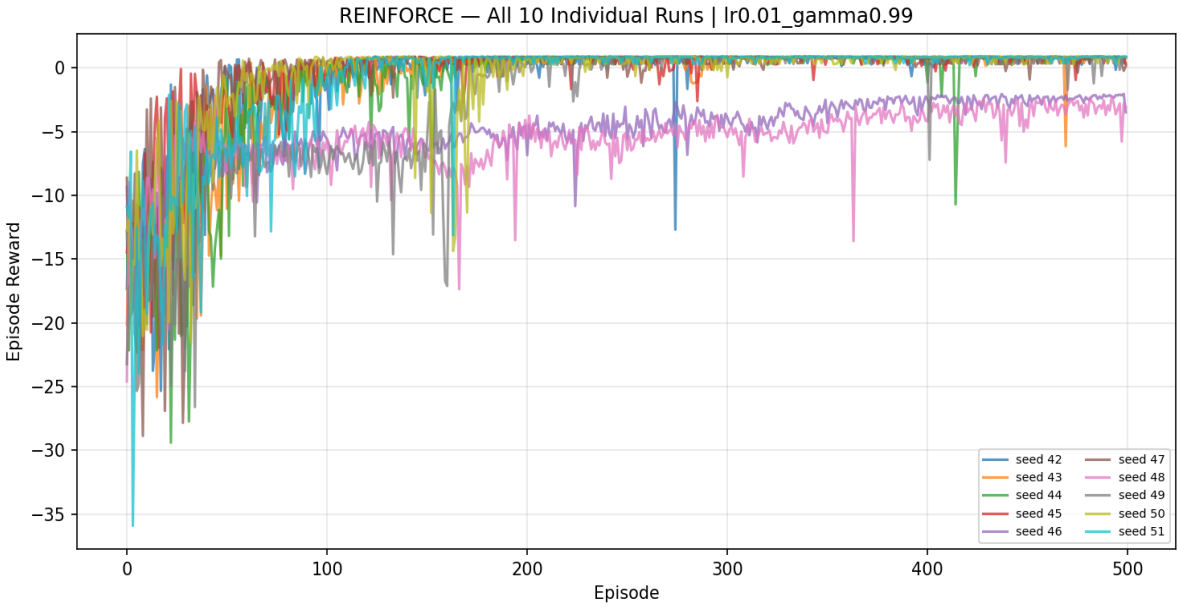

Figure 3. All 10 individual REINFORCE seed runs (lr=0.01, γ=0.99). Note seeds 46 and 48 exhibit persistently lower performance.

Figure 3 shows considerably more heterogeneity. While the majority of seeds converge to near-zero reward by episode ~200, seeds 46 and 48 remain notably lower (around –3 to –5) even at episode 500. This bimodal behaviour is consistent with the known instability of REINFORCE: the policy gradient estimator has high variance, and the Monte Carlo return estimator is further inflated by the 10% action noise of the environment. If an episode happens to explore an unfortunate sub-region early on, the policy update pushes the network weights in a direction that is difficult to escape, particularly when combined with a relatively large discount factor (0.99) that amplifies long-return credit assignment errors. This variability cannot be assessed from the mean alone; the individual-run plot is essential for identifying that the algorithm's mean performance masks a minority of failed runs.

4.3 Hyperparameter Sensitivity

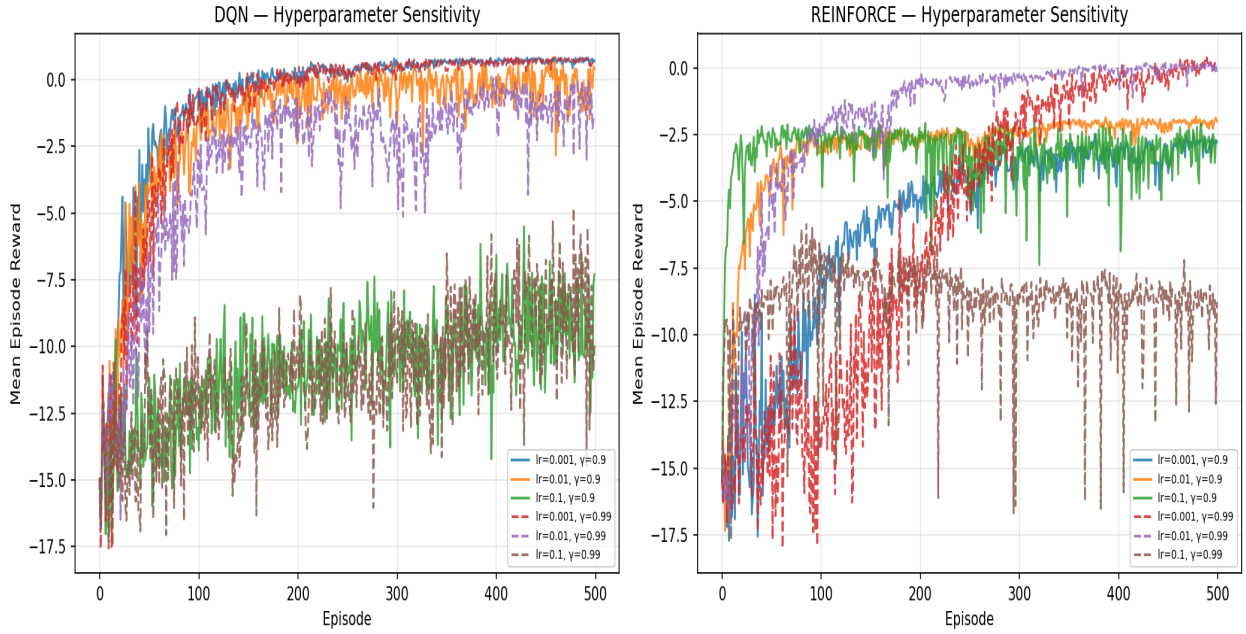

Figure 4. Mean episode reward for all 6 hyperparameter configurations (3 LR × 2 γ) for DQN (left) and REINFORCE (right).

DQN (Figure 4, left)

DQN is highly sensitive to learning rate. lr=0.001 with both γ values converges cleanly to near-zero reward and is clearly the best configuration. lr=0.01 produces mixed results: with γ=0.9 most seeds converge reasonably well, but with γ=0.99 several seeds exhibit catastrophic forgetting (mean reward drops to –8 or lower), likely due to overestimation of long-horizon Q-values combined with a large learning step. lr=0.1 completely fails to learn for both discount factors, producing mean rewards near –11, comparable to or worse than the random baseline. At lr=0.1, the Bellman updates overshoot the Q-function surface, destabilising the target network and causing the weights to oscillate. The best DQN configuration is therefore **lr=0.001, γ=0.99**, which combines slow, stable value propagation with long-term discounting that incentivises reaching the distant goal.

REINFORCE (Figure 4, right)

REINFORCE shows a different pattern. lr=0.01 with γ=0.99 achieves the best mean performance, closely followed by lr=0.1 with γ=0.9. The very low learning rate (0.001) converges only partially within 500 episodes regardless of discount factor, suggesting that REINFORCE needs a moderately-sized gradient step to overcome the high noise in the Monte Carlo return estimator. Interestingly, lr=0.1 with γ=0.99 is the *worst* REINFORCE configuration (mean reward ~–10), demonstrating that combining a large step size with a large discount factor amplifies return variance to the point of policy collapse. The best configuration is **lr=0.01, γ=0.99**.

Summary: Best Hyperparameter Configurations

Algorithm	Best lr	Best γ	Mean Reward (final 50 ep)	Notes

DQN	0.001	0.99	~0.0	Stable; tight seed agreement
REINFORCE	0.01	0.99	~-0.9	2 seeds lag; higher variance
Random	N/A	N/A	~-14.4	Lower bound baseline

5. Policy Visualisation

Policy heatmaps show the greedy action (arrow direction) in each free cell, overlaid with a visit-count heatmap from a 1,000-step greedy rollout. Obstacles are black; the goal is green; the start is marked with a star.

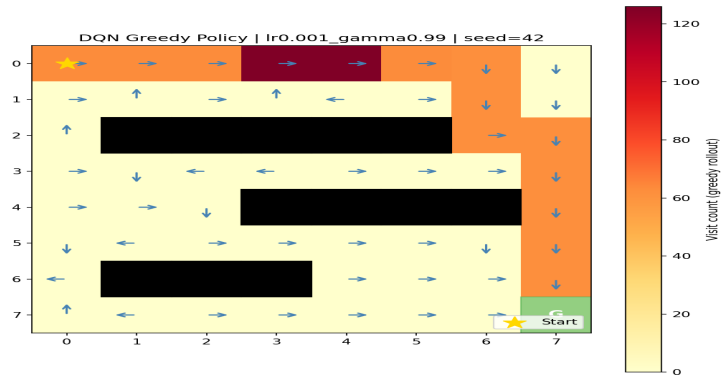

Figure 5. DQN greedy policy (lr=0.001, γ=0.99, seed=42).

The DQN policy (Figure 5) shows a clear, coherent routing strategy. The agent exits the start cell immediately to the right along row 0, threading through the gap in the first obstacle row (cols 0 or 6), then descending along column 7 toward the goal. The visit count heatmap is concentrated in a narrow corridor along the top row and right column, confirming the agent has learned a deterministic near-optimal path that avoids all three obstacle rows. This tight path is consistent with DQN's off-policy nature: the replay buffer allows it to efficiently refine Q-values across the entire state space, leading to a globally coherent policy.

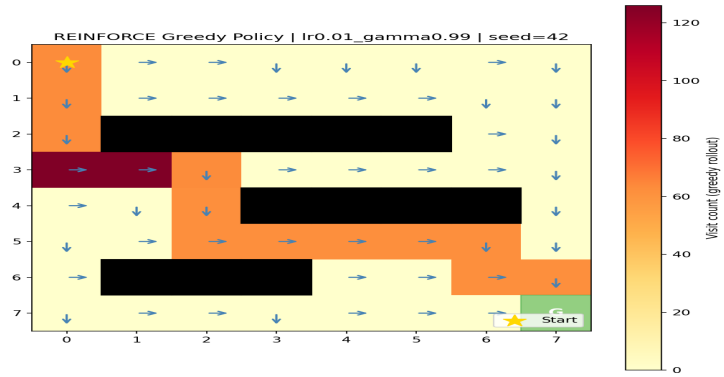

Figure 6. REINFORCE greedy policy (lr=0.01, γ=0.99, seed=42).

The REINFORCE policy (Figure 6) takes a different but also valid route: it descends along column 0 to bypass obstacle row 2 from the left, then moves right along row 3 to navigate through the gap in obstacle row 4, ultimately approaching the goal from the left. The visit-count heatmap is more spread out, reflecting the stochastic nature of on-policy exploration: REINFORCE never uses a replay buffer, so the policy only improves along trajectories it has recently visited. This produces a slightly less compact but still functionally correct policy that reliably reaches the goal.

6. Reflection

6.1 Mean Performance vs. Seed Variability

For DQN, mean and variability are tightly aligned: the mean converges cleanly and the individual seed plots confirm that this is representative of every run. The standard error is small in the converged region, meaning the mean is a reliable summary statistic. For REINFORCE, the mean obscures important structure. Seeds 46 and 48 remain at approximately –3 to –5 even at episode 500, pulling the mean down relative to the eight seeds that converge well. The high variability arises from two compounding factors: (1) Monte Carlo return estimates have variance proportional to episode length, which is long in this maze environment; and (2) the 10% stochastic action noise further inflates the variance of the policy gradient estimator. To better assess this variability one could: run more seeds (e.g. 30+) to obtain tighter confidence intervals; apply the interquartile range instead of standard error as a robust spread measure; or test whether a baseline (e.g. value function baseline / actor-critic) reduces variance sufficiently to eliminate the bimodal failure mode.

6.2 JAX Functional Style

JAX's functional paradigm had a direct and beneficial impact on the implementation. The requirement that JIT-compiled functions be pure (no side effects, immutable state) enforced a clean separation between the stateful parts of training (environment interaction, replay buffer, Python loop) and the stateless computational kernel (Bellman update, policy gradient step). This made it natural to factor the update logic into a factory function (*make_dqn_update_fn*, *make_reinforce_update_fn*) that closes over the optimizer and gamma, producing a single compiled binary reused for all episodes. **vmap** was used to batch the MLP forward pass without writing explicit loop code, which both simplified the code and enabled automatic vectorisation on the GPU. The main limitation of JAX in this setting is that the environment (Gymnasium) is stateful and not JAX-native, requiring a boundary between NumPy-based environment stepping and JAX-based parameter updates. Handling this boundary via **jnp.asarray()** conversions at each step added minor overhead but kept the code straightforward.

6.3 Environment Design and Research Questions

The maze structure was effective in surfacing the algorithmic differences of interest. The sparse +1 goal reward ensured that credit assignment was genuinely challenging: a random agent almost never reaches the goal, confirming that the maze difficulty is appropriate. The three staggered obstacle rows created distinct corridor bottlenecks, which is why DQN and REINFORCE learned qualitatively different paths (top-right corridor vs. left-side descent) despite both succeeding. The 10% action noise successfully increased REINFORCE variability without destabilising DQN, supporting the hypothesis that off-policy methods with replay buffers are inherently more robust to environment stochasticity than on-policy Monte Carlo methods.

6.4 GPU Usage Details

All experiments were submitted to the University of Ottawa Morningstar HPC cluster as a single SLURM job (job-4018) requesting one GPU under the *small* partition with a 24-hour wall time. The

environment was a Python 3.10 virtualenv with JAX, Optax, Gymnasium, NumPy, and Matplotlib. Total wall-clock time for the full sweep (6 configurations × 10 seeds for both DQN and REINFORCE, plus the random baseline) was approximately 5.3 hours. DQN runs were fastest (~27–165 seconds/run depending on lr) while REINFORCE runs were significantly slower (~68–496 seconds/run) due to the need to collect full episodes before each gradient update. High-lr DQN and REINFORCE runs were generally slower because unstable training led to longer episodes before truncation at 200 steps.

6.5 AI Tool Usage

Claude (Anthropic) was used during development of this assignment. Specifically, the AI was consulted for: (1) structuring the JAX factory pattern for JIT-compiled update functions; (2) identifying the correct use of **jax.lax.stop_gradient** for the DQN target network; and (3) debugging a numerical instability in the REINFORCE action sampling where probabilities did not sum exactly to 1.0 after log-softmax due to floating point precision, resolved by explicit renormalisation. All final code decisions, hyperparameter choices, environment design, and experimental analysis were made independently by the author. The AI was not used to generate the environment design rationale or the results analysis.

7. Conclusion

Both DQN and REINFORCE successfully learned to navigate the stochastic 8×8 maze from a sparse reward signal, vastly outperforming the random baseline. DQN achieved faster and more consistent convergence across all seeds, confirming the advantage of off-policy learning with experience replay in environments with sparse rewards and stochastic dynamics. REINFORCE converged more slowly and exhibited greater seed-to-seed variability, with two of ten seeds showing persistently sub-optimal policies that the mean statistic alone would have concealed. The best hyperparameter configuration for both algorithms was lr=0.001 (DQN) or lr=0.01 (REINFORCE) with γ=0.99, confirming that high discount factors are necessary to propagate the sparse goal reward over long paths. JAX's functional design imposed a clean architecture and enabled efficient GPU execution via JIT compilation and vmap vectorisation.